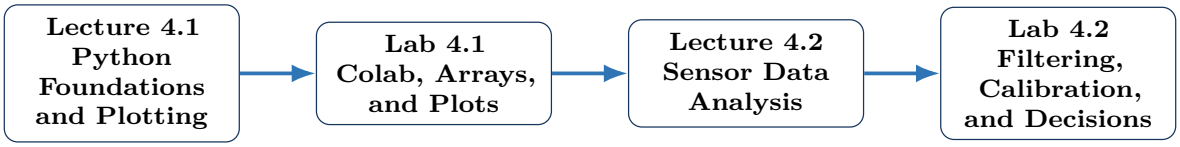
Week 4: Python for Mechatronics

Google Colab, Engineering Plots, and Sensor Data Analysis

ENGR 1105 | Fall 2026



Week 4 Goal

Use free browser-based Python tools to organize engineering data, perform calculations, create readable plots, filter noisy measurements, calibrate a sensor, quantify error, and convert measurements into useful operating states.

Opening Questions

1. How can a graph reveal a pattern that is hard to see in a Serial Monitor?
2. Why should we keep raw data even after filtering?
3. How could we decide whether a sensor reading means DANGER, WARNING, or SAFE?

Lecture 4.1

Python Foundations and Engineering Plots

Week 4

Learning Objectives

1. explain why Python is useful for robotics and mechatronics;
2. use Google Colab code and text cells;
3. create variables, lists, and NumPy arrays;
4. use zero-based indexing and slicing;
5. calculate basic statistics;
6. create engineering plots with Matplotlib; and
7. use decisions, loops, and functions.

1. Why Python?

Python is a general-purpose programming language used widely in engineering, robotics, data science, automation, and scientific computing. In this course we will use **Google Colab**, a browser-based notebook environment, so students do not need to purchase software or install Python.

Python Is a Tool for Engineering Reasoning

Python makes repetitive calculations and data visualization easier. The engineer still chooses the correct units, equations, assumptions, thresholds, and interpretation.

2. Google Colab and Jupyter Notebooks

A notebook combines **code cells**, **text cells**, numerical output, tables, and plots in one document.

Notebook element	Purpose
Code cell	Run Python commands and calculations.
Text cell	Record headings, explanations, equations, and conclusions.
Files pane	Upload CSV files used by the notebook.
Runtime	The temporary computer that executes the notebook code.
Output area	Displays numbers, tables, messages, and plots.

Save Your Work

Colab runtimes are temporary. Save the notebook in Google Drive or download the `.ipynb` file before leaving class. Uploaded files may need to be uploaded again after a runtime reset.

3. Your First Python Cell

```
voltage = 5.0
current = 0.12
power = voltage * current
print(power)
```

Python uses = for assignment. A semicolon is not required at the end of a Python statement.

4. Variables and Data Types

```
student_count = 12        # integer
voltage = 4.85            # floating-point number
motor_on = True           # Boolean
state = "SAFE"            # string
```

Use descriptive names such as `raw_distance_cm`, not names such as `x1` when the physical meaning matters.

5. Lists and NumPy Arrays

A Python list stores a sequence. A NumPy array adds efficient numerical operations.

```
import numpy as np

readings_list = [20.6, 20.1, 19.8, 20.4, 20.2]
readings = np.array(readings_list)

print(readings)
```

Why NumPy?

NumPy arrays let us perform calculations on an entire vector of measurements without writing a loop for every value.

6. Zero-Based Indexing and Slicing

Python starts counting at index 0.

```
readings[0]      # first value
readings[2]      # third value
readings[-1]     # last value
readings[1:4]    # indices 1, 2, and 3
```

> **Python Starts at 0**
>
> The first element has index 0. In a slice `a:b`, index `a` is included and index `b` is not included.

7. Vectorized Calculations

```
voltage = np.array([1.0, 2.0, 3.0, 4.0])
current = np.array([0.10, 0.12, 0.14, 0.16])
power = voltage * current
voltage_squared = voltage ** 2
```

For NumPy arrays, multiplication and exponentiation are automatically element by element.

8. Useful Statistics

```
print(np.min(readings))
print(np.max(readings))
print(np.mean(readings))
print(np.std(readings))
print(len(readings))
```

Mean describes the center of repeated measurements. Standard deviation describes spread.

9. Pandas DataFrames

Pandas organizes labeled tabular data, similar to a spreadsheet.

```
import pandas as pd

df = pd.DataFrame({
    "time_s": [0, 1, 2],
    "distance_cm": [20.6, 20.1, 19.8]
})

print(df)
```

Columns can be selected by name:

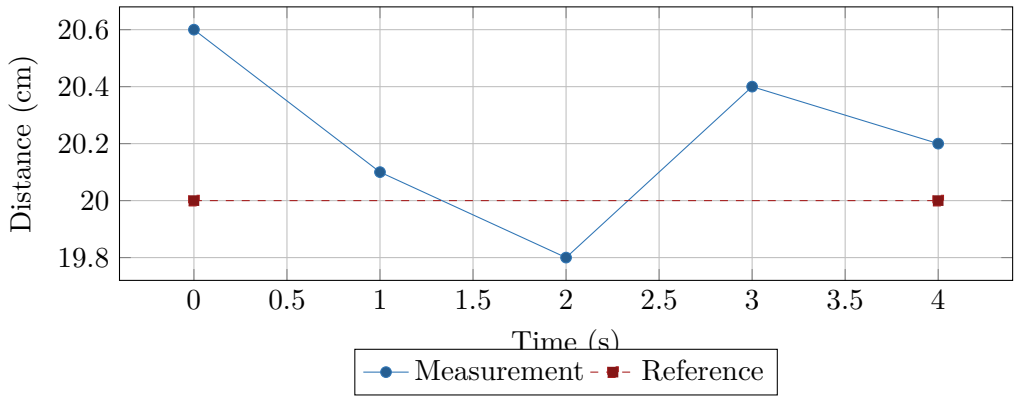
```
distance = df["distance_cm"]
```

10. Engineering Plots with Matplotlib

```
import matplotlib.pyplot as plt

plt.plot(df["time_s"], df["distance_cm"], "o-")
plt.xlabel("Time (s)")
plt.ylabel("Distance (cm)")
plt.title("Ultrasonic Sensor Reading")
plt.grid(True)
plt.show()
```

A useful engineering plot needs a descriptive title, axis labels, units, readable data markers or lines, and a legend when multiple data sets are shown.



11. Multiple Data Sets

```
plt.plot(time, raw, "o-", label="Raw")
plt.plot(time, filtered, "-", label="Filtered")
plt.legend()
```

12. Decision Statements

```
if distance_cm < 15:
    state = "DANGER"
elif distance_cm < 35:
    state = "WARNING"
else:
    state = "SAFE"
```

Indentation is part of Python syntax. All statements belonging to the same decision block must line up.

13. Loops

```
for value in readings:
    print(value)
```

Loops are useful for repeated procedures, but NumPy and pandas often let us process entire columns directly.

14. Simple Functions

```
def adc_to_voltage(adc_value, v_ref=5.0):
    return adc_value * v_ref / 1023.0

voltage = adc_to_voltage(512)
```

Functions make programs easier to test, reuse, and explain.

Lecture 4.1 Check

1. What index accesses the first element of a Python list?
2. When would you use a NumPy array instead of a basic list?
3. Name four features of a good engineering plot.
4. What does indentation communicate in an `if` statement?

Lecture 4.2

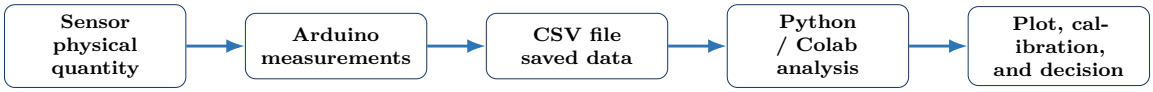
Sensor Data Analysis and Reliable Decisions

Week 4

Learning Objectives

1. import an Arduino CSV file with pandas;
2. inspect and clean measurements;
3. filter data with a moving average;
4. fit a linear calibration model;
5. calculate measurement error;
6. classify measurements into operating states; and
7. communicate results with tables and plots.

15. From Arduino to Python



In Week 3, Arduino produced sensor readings. In Week 4, Python helps us turn a sequence of readings into evidence.

16. Upload and Import a CSV File

In Colab, upload the provided `sample_sensor_data.csv` file using the Files pane, then run:

```
import pandas as pd

df = pd.read_csv("sample_sensor_data.csv")
print(df.head())
```

The example file contains `time_s`, `raw_distance_cm`, and `reference_distance_cm`.

17. Inspect the Data First

```
print(df.head())
print(df.tail())
print(df.shape)
print(df.describe())
```

Never start filtering or fitting a model before checking column names, units, missing values, and obviously invalid values.

18. Valid and Invalid Measurements

For an HC-SR04-style classroom sensor, a zero value in our data file represents an invalid reading.

```
valid = (
    (df["raw_distance_cm"] > 1) &
    (df["raw_distance_cm"] < 400)
)

clean = df.loc[valid].copy()
```

The Boolean mask `valid` contains `True` or `False` for each row.

19. Moving Average Filtering

A moving average replaces a measurement with the average of nearby values.

```
clean["filtered_cm"] = (
    clean["raw_distance_cm"]
    .rolling(3, center=True, min_periods=1)
    .mean()
)
```

A wider window usually produces a smoother signal but can hide fast changes.

Filtering Tradeoff

Smoothing reduces short random variation, but too much smoothing can delay or erase real changes. Choose a filter window based on the physical system.

20. Compare Raw and Filtered Data

```
plt.plot(clean["time_s"], clean["raw_distance_cm"],
         "o-", label="Raw")
plt.plot(clean["time_s"], clean["filtered_cm"],
         "-", label="Filtered")
plt.xlabel("Time (s)")
plt.ylabel("Distance (cm)")
plt.legend()
plt.grid(True)
plt.show()
```

21. Calibration with a Linear Model

A simple calibration model is

$$y = mx + b,$$

where x is the raw sensor reading and y is the reference value.

```
import numpy as np

m, b = np.polyfit(
    clean["raw_distance_cm"],
    clean["reference_distance_cm"],
    1
)

clean["calibrated_cm"] = (
    m * clean["raw_distance_cm"] + b
)
```

22. Evaluate Error

```
clean["raw_error_cm"] = (
    clean["raw_distance_cm"] -
    clean["reference_distance_cm"]
)

clean["cal_error_cm"] = (
    clean["calibrated_cm"] -
    clean["reference_distance_cm"]
)

raw_mae = np.mean(np.abs(clean["raw_error_cm"]))
cal_mae = np.mean(np.abs(clean["cal_error_cm"]))
```

Mean absolute error (MAE) summarizes the typical size of the error without positive and negative errors canceling each other.

23. Calibration Is Evidence-Based

A calibration model is useful only if it improves performance over the operating range that matters. Always compare raw and calibrated errors rather than assuming calibration helped.

24. Classifying Sensor States

```python
def classify(distance_cm):
    if distance_cm < 15:
        return "DANGER"
    elif distance_cm < 35:
        return "WARNING"
    return "SAFE"

clean["state"] = clean["filtered_cm"].apply(classify)
```

The state labels are engineering decisions based on thresholds, not new sensor measurements.

25. Results Table

```python
results = clean[[
    "time_s",
    "raw_distance_cm",
    "filtered_cm",
    "calibrated_cm",
    "state"
]]

print(results.round(2))
```

26. Complete Analysis Workflow

1. import packages;
2. upload/read the data file;
3. inspect data;
4. mark invalid readings;
5. calculate filtered values;
6. fit a calibration model;
7. calculate error;
8. classify operating states;
9. create plots and tables;
10. write an engineering conclusion.

27. Compact Example

```python
import numpy as np
import pandas as pd
```

```
import matplotlib.pyplot as plt

df = pd.read_csv("sample_sensor_data.csv")
valid = (df.raw_distance_cm > 1) & (df.raw_distance_cm < 400)
clean = df.loc[valid].copy()

clean["filtered_cm"] = clean.raw_distance_cm.rolling(
    3, center=True, min_periods=1).mean()

m, b = np.polyfit(clean.raw_distance_cm,
                  clean.reference_distance_cm, 1)
clean["calibrated_cm"] = m*clean.raw_distance_cm + b
```

28. Engineering Communication

A strong Week 4 analysis should include:

- the question being investigated;
- units and test conditions;
- a readable plot;
- the filtering method and window size;
- the calibration equation;
- an error metric;
- the decision thresholds; and
- a conclusion that refers to evidence.

29. Common Python / Colab Problems

Problem	First check
`FileNotFoundError`	Upload the CSV and confirm the exact file name.
`NameError`	Run the earlier cell that creates the variable or import.
Plot is blank or wrong	Check selected columns, units, and cell execution order.
`KeyError` for a column	Print `df.columns` and check spelling.
Code stops after runtime reset	Rerun import/setup cells and upload data again.
Results changed unexpectedly	Run cells from top to bottom so variables are recreated consistently.

Do Not Trust a Plot Automatically

A graph can be misleading if units, axes, filtering choices, or invalid data are wrong. Always connect the plot back to the physical experiment.

30. Week 4 Summary

Core Ideas

- Python and Google Colab provide a free environment for engineering computation.
- NumPy arrays support numerical calculations; pandas DataFrames organize labeled data.
- Python uses zero-based indexing.
- Matplotlib creates engineering plots.
- Boolean masks remove invalid measurements without deleting the raw file.

- Rolling averages can reduce noise.
- `np.polyfit()` can estimate a simple calibration line.
- Error metrics help evaluate whether calibration improved the sensor.
- Thresholds convert measurements into controller states.

Lecture Exit Questions

1. Why do we keep the raw data after filtering?
2. What does a three-sample moving average do?
3. What do m and b mean in a linear calibration model?
4. Why should raw and calibrated errors be compared?
5. What is the difference between a measured value and a state label?